

Mathematics Extended Essay on the topic of Erdős’
Probabilistic method

Candidate Number:

May 31, 2026

Word count: TO UPDATE

Contents

1	Introduction	2
2	Combinations and Permutations	2
3	Scale Free Networks and Graph Theory	3
3.1	Definitions	4
3.2	Scale Free Networks	4
4	The Probabilistic Method	5
5	Predicting clique sizes in Scale Free Networks	5
6	Building Scale Free Networks	5
7	Comparing Results	5
8	Conclusion	5

How can we use Erdős' Probabilistic method to explore clique sizes in Scale-Free Networks without requiring constructive methods?

1 Introduction

- A brief introduction - My motivation to explore this topic

In my endless search for a fitting topic to write my Maths Extended Essay in, I came across the branch of combinatorics. Never having understood the underlying field of study, and knowing that we would cover very little of it in the AI HL syllabus, I was interested in exploring it further.

To briefly help the reader to understand this essay better, below are a few definitions and pointers on notation which might be useful throughout the next few pages:

Object The concept of an object is a highly philosophical matter, yet for the purpose of this essay, the reader can assume that it is simply an abstract *thing*. It can be anything from a number, a node of a graph or a name on a list.

Set A set is a collection of such *things*, i.e. objects.

Element An element is a distinct object in a set.

$s \in S$ This notation ($\in$) will come up frequently and means "s is an element of S".

Binomial Coefficient The notation of a binomial coefficients will be discussed in the next section.

2 Combinations and Permutations

To understand the rest of this essay, it is important to clarify the concept of what are colloquially known as *combinations*. Imagine a simple cake in which all ingredients are mixed independent of the order. In mathematical language, that is called a combination of objects. If, on the other hand, the order matters, like for a phone password, that is called a permutation. There is a further distinction between a combination and permutation where repeats are allowed or not. A phone password, for example, can have repeating digits, arranging the winners of a race, on the other hand, cannot.

Calculating possible arrangements of objects depends on the requirements. It is simplest to calculate all possible permutations when repetitions are allowed. If we have a 4-digit phone password where every digit can be a digit from 0-9 with repetitions there are:

$$10 \times 10 \times 10 \times 10 = 10^4 = 10000$$

possible permutations since every new digit we can choose from another set of 10 numbers.

Generalising, if we have n different types in a specific order of r items with repetitions allowed, we can assume there are:

$$n^r \tag{1}$$

permutations.

If, however, we cannot have repeating values, because we are working with all the possible results of a race, we have to decrease the pool of objects to pick from with every step. When repetitions were allowed, we could multiply the same number over and over.

This time, however, we need to calculate the possibilities from a decreasing number of choices. Taking 7 runners in a race, there are 7 options to be first, but then only 6 to come second, then 5 and so on. There are therefore

$$7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 =$$

possibilities. Mathematically, this can be denoted as $7!$. If want to choose the first three winners only we would multiply $7 \times 6 \times 5$. Since our notation goes until 1 however, we can cancel out the latter part:

$$\frac{7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1}{4 \times 3 \times 2 \times 1} = \frac{7!}{4}$$

Generalising, if we chose r items from n different types where repetitions are not allowed we get:

$$\frac{n!}{(n-r)!} \quad (2)$$

If we apply this idea to combinations where there are no repetition, in other words we don't care about the order but cannot have duplicates, we need to remove all duplicates. Taking our race example, if we only care about the all the possible lists we can generate, independent of their order, we have to remove all the ways in which 3 objects (the value from before) can be arranged. As a reminder, that is $3!$. Now applying that to the formula:

$$\frac{\text{number of possible combinations}}{\text{cancelling out} \times \text{eliminating duplicates}} = \frac{7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1}{(4 \times 3 \times 2 \times 1) \times (3 \times 2 \times 1)} = \frac{7!}{4! \times 3!}$$

Conventional notation switches the latter around meaning we generalise to:

$$\frac{n!}{r!(n-r)!}$$

This formula is so common that it is shortened to:

$$\frac{n!}{r!(n-r)!} = \binom{n}{r} \quad (3)$$

The notation means " n chooses r " and we are taking r items from a pool of n different types where the order does not matter, but there can't be repeats. This formula is called the binomial coefficient and will be used frequently throughout this essay.

3 Scale Free Networks and Graph Theory

- Describing the concept of graphs - Defining terms - Applying the idea to scale free networks

One predominant branch of Mathematics which is often ignored is called combinatorics. It focuses on the idea of counting, which is applied in order to find a result but can also define a final answer. I will talk more about that concept using an example below. Part of combinatorics is the concept of graphs which is related to graph theory which explores the relationship between objects. These should not be confused with a graph on a Cartesian plane. These graphs allow the mathematical analysis of networks and connections, like streets, mobile networks, or social media networks. From now on the use of the word graph will refer to the mathematical graph which makes part of discrete mathematics, i.e. a network of nodes connected by edges.

3.1 Definitions

For clarity, I will define the following terms, as they are different around the mathematical world:

Node: A node is the same as a vertex and is the undivisible point at which one or more edges meet.

Edge: An edge is the connection between nodes.

Degree: The degree of a node is the number of connections, i.e. edges, it has to another node. In general, this essay will denote it as k to simplify understanding.

Complete Graph A graph in which every pair of distinct nodes is connected by an edge. It will be denoted as K_n where n is the number of nodes in the graph.

Subgraph A subgraph by which is most commonly meant induced subgraph is a subset of nodes of a graph and all the edges connecting pairs of the subset. This creates a new graph "inside" another graph.

Clique A clique is a tighter definition of a subgraph as it requires the subgraph to be complete.

I am only making use of simple undirected graphs, meaning that there can only be a single edge between every node and that the edges don't represent a direction in the connection between nodes (as would be required when representing one-way streets for example).

3.2 Scale Free Networks

This extended essay will investigate scale free networks which are representative of many real-world networks because of their "Hubs". A scale free network gives a power law relation between the degree of a node and the probability of its occurrence. In other words,

$$P_{(deg)}(k) \propto k^{-\gamma}$$

where γ is some exponent and $P_{(deg)}(k)$ the probability of a node with degree k .

To generate scale free networks, different methods exist. The greater category of the most commonly used model is called *configuration model* because it allows the degree of every node to be pre-defined. Since scale free networks are defined by their degree separation that is a requirement. When it is of interest to fix the degree sequence exactly it is possible to make use of a microcanonical model which defines the exact degree of every node in the graph and randomly draws edges according to these requirements. On the other hand, a canonical model works with the probability distributions of the edges in a network. The one this extended essay focuses on is the Chung-Lu (canonical) Configuration Model because the probabilistic generation is also easier to analyse using the probabilistic method.

In short, the model defines the probability of an edge between two nodes h and j to be:

$$p_{hj} = \frac{k_h k_j}{\sum_i k_i} = \frac{k_h k_j}{2m} \quad (4)$$

where p_{hj} is the probability, k_h and k_j are their respective degrees (remember: the number of nodes they are connected to). We divide by the sum of all degrees to make a probability that sits between 0 and 1.

Equation (4) comes from the bounds set by the definitions of algebraic mathematics and graph theory. Chung & Lu explicitly define the graph $G(\mathbf{w})$ where $\mathbf{w}$ is the expected degree sequence. The expected degree sequence of a graph is the decreasing sequence of its nodes' degrees. In the graph below that would be:

They further define that $G \in G(\mathbf{w})$ to show that we are not talking about a subgraph. To make the model work, it is further stated that all probabilities are independent and that

4 The Probabilistic Method

- Describing Probabilistic Notation - Applying using simple a simple Ramsey number example ($R(3,3) = 6$)

As we discussed above, graph theory is computationally difficult to explore. When trying to generate every possible combination of words

5 Predicting clique sizes in Scale Free Networks

- Setting out the problem: we want to find specific clique sizes in a scale free network with 4 hubs - Using the Chung-Lu Random Graph Model, we will describe how to statically generate a graph, with different probabilities of nodes and edges occurring. - Using the first moment we will explain the concept of

6 Building Scale Free Networks

- Using the help of AI to code a python script, I aim to generate a statistically meaningful number of Scale-Free Networks. - By analysing the sizes of clique's present, I should be able to find similar results to my theoretical/probabilistic approach

7 Comparing Results

- Comparing results and making sure theory aligns with empirical data

8 Conclusion

- Conclude on the uses of my results